SQL SERVER & AZURE SQL DATABASE

DBA Training Program

Trainees Handout

12 Weeks | Complete Beginners | DP-300 Aligned | v2.0

Month 1: SQL Foundations • Month 2: Core DBA Operations • Month 3: Advanced DBA & Cloud

Tools: SQL Server Management Studio (SSMS) | Azure Portal | Azure Data Studio/VS Code+ SQL Plugins
Sandboxes: SQL Server 2019 on Windows Server 2019 + Azure SQL Database (Free Tier)

Welcome to the Program

Over the next 12 weeks you will go from knowing nothing about SQL to being able to perform core database administration tasks on both SQL Server 2019 and Azure SQL Database. Every skill you build maps directly to real DBA work — and to the DP-300: Administering Relational Databases on Microsoft Azure certification exam.

Your two sandbox environments — use both throughout the Program:

- SQL Server 2019 on your Windows Server 2019 VM — your simulated on-premises server
- Azure SQL Database (Free Tier) in the Azure Portal — your cloud database

Always know which environment you are working in. The T-SQL language is nearly identical, but the management tools, security model, and backup behavior are different.

Your 12-Week Roadmap

Month	Week	Topic & Badge
Month 1: SQL Foundations	**Wk 1**	Your First Database — SQL basics, SSMS, SELECT queries
	Wk 2	Speaking SQL — JOINs, aggregates, DML, transactions
	Wk 3	Building the Blueprint — DDL, constraints, indexes, schemas
	Wk 4	Lock It Down — Security, logins, users, permissions
Month 2: Core DBA Ops	**Wk 5**	Never Lose a Byte — Backup & restore, recovery models
	Wk 6	Data Wrangler — CSV imports, BULK INSERT, BCP
	Wk 7	Launch to the Cloud — On-prem to Azure migration
	Wk 8	Speed Matters — Performance monitoring, Query Store
Month 3: Advanced DBA	**Wk 9**	The Robot DBA — SQL Agent automation, Elastic Jobs
	Wk 10	Bulletproof Database — High availability & disaster recovery

	Wk 11	☁ Eyes on the Cloud — Azure monitoring & operations
	Wk 12	🏆 The Grand Finale — Full capstone & DP-300 countdown

⏱ Time Budget

- 2–3 hours: Watch the free videos and complete the Microsoft Learn modules
- 2–4 hours: Complete the hands-on lab tasks in your sandboxes
- 30–60 minutes: Tackle the weekly Teams challenge
- 30 minutes (optional): Help a teammate — teaching is the fastest way to deepen your own understanding

WEEK 1

Your First Database

SQL Fundamentals, SSMS Setup & Basic SELECT Queries

⏱ 5–10 hrs / week | 🛠 SSMS + Azure Portal

📌 Learning Objectives

- Understand what a relational database management system (RDBMS) is
- Know the difference between SQL Server 2019 (on-prem) and Azure SQL Database (cloud)
- Install SSMS and connect to both sandbox environments
- Navigate the SSMS Object Explorer: databases, tables, schemas, views
- Write SELECT queries using WHERE, ORDER BY, TOP, and DISTINCT
- Handle NULL values with ISNULL(), IS NULL, IS NOT NULL

📖 Key Concepts

SQL Server vs Azure SQL — same language, different world: SQL Server 2019 runs on a machine you manage. Azure SQL Database is fully managed by Microsoft — they handle backups, patching, and high availability. Both use T-SQL, which is why learning them together is so powerful.

Essential T-SQL this week:

- SELECT col1, col2 FROM schema.Table — retrieve data
- WHERE col = 'value' | WHERE price > 100 — filter rows
- ORDER BY col ASC / DESC — sort results
- TOP (n) — return only the first n rows
- DISTINCT — remove duplicate values from results
- ISNULL(col, 'default') — replace NULL with a fallback

Common data types: INT, BIGINT, VARCHAR(n), NVARCHAR(n), DATETIME, DATE, DECIMAL(p,s), BIT, UNIQUEIDENTIFIER

🔗 Free Learning Resources

Resource	What You'll Learn
🛠 Download SSMS	Install this first — it is your primary tool for the entire Program.
📺 WiseOwl: SQL Server for Beginners (Episodes 1–5)	The best free SQL Server beginner playlist on YouTube. Watch episodes 1–5 this week. Search: 'WiseOwl SQL Server Beginners'.

MS Learn: Introduction to T-SQL	~1 hour free official module with browser-based exercises. Do this before the lab.
MS Learn: Azure SQL Fundamentals (Module 1)	Start Module 1 only this week. Covers what Azure SQL is and how to connect.
Download AdventureWorks2019 Backup	Your primary practice database. Download AdventureWorks2019.bak.

Hands-On Lab

Note: Before connecting to Azure SQL from SSMS, go to the Azure Portal → your SQL server → Networking → add your client IP address as a firewall rule. Without this step the connection will silently fail.

Complete all tasks and send screenshots of your results to your mentor each week.

1. Restore AdventureWorks2019.bak to your local SQL Server: in SSMS right-click Databases → Restore Database → Device → add the .bak file → OK
2. Connect to your Azure SQL Database sandbox in SSMS using the server FQDN (yourserver.database.windows.net) and SQL auth credentials from the Azure Portal
3. Verify your firewall rule is set in the Azure Portal (Server → Networking → add your IP) before step 2
4. Write a SELECT query returning all first names and last names from Person.Person
5. Write a query returning only products from Production.Product where ListPrice > $500 — how many rows do you get?
6. Write a query showing the TOP 10 most expensive products ordered by ListPrice descending — show Name and ListPrice
7. Use ISNULL() on the Color column of Production.Product to replace any NULL values with the text 'Not Specified'

DP-300 Alignment: Section 1.1 — Plan and implement Azure SQL Database resources

WEEK 2

Speaking SQL

JOINs, Aggregates, DML & Transactions

⏱ 5–10 hrs / week | 🛠 SSMS + Azure Portal

Learning Objectives

- Join two or more tables using INNER JOIN and LEFT JOIN
- Use aggregate functions: COUNT(), SUM(), AVG(), MIN(), MAX()
- Group and filter aggregates with GROUP BY and HAVING
- Modify data safely with INSERT, UPDATE, DELETE, and TRUNCATE
- Understand and use transactions: BEGIN TRAN, COMMIT, ROLLBACK

Key Concepts

JOINs — connecting tables: Most useful data lives across multiple tables. INNER JOIN returns only rows that match in both tables. LEFT JOIN returns all left-table rows plus NULLs where the right table has no match.

The golden rule of DML: Always use a WHERE clause on UPDATE and DELETE. Without it, every row in the table is affected. Wrap all DML in a transaction (BEGIN TRAN) so you can ROLLBACK if something goes wrong.

UPDATE NEEDS WHERE AND BEGIN TRAN SO WE CAN ROLLBACK INCASE

T-SQL this week:

- INNER JOIN tB ON tA.col = tB.col — matching rows only
- LEFT JOIN — all left rows; NULL where no right match
- COUNT(*), SUM(col), AVG(col), MIN(col), MAX(col)
- GROUP BY col HAVING COUNT(*) > 5 — filter aggregated groups
- INSERT INTO table (c1,c2) VALUES ('a', 1)
- UPDATE table SET col = val WHERE id = 1
- DELETE FROM table WHERE id = 1
- BEGIN TRAN / COMMIT TRAN / ROLLBACK TRAN

Free Learning Resources

Resource	What You'll Learn
📺 WiseOwl: SQL Server Beginners (Episodes 6–10)	Continue the WiseOwl playlist — episodes 6–10 cover JOINs and aggregates. Search: 'WiseOwl SQL Server Beginners'.

MS Learn: Join Multiple Tables	~45 min free module on INNER and OUTER JOINs with interactive exercises.
MS Learn: Write T-SQL Queries	Covers aggregates, GROUP BY, and HAVING in detail.

Hands-On Lab

Complete all tasks and send screenshots of your results to your mentor each week.

1. Write an INNER JOIN between Person.Person and HumanResources.Employee — show each employee's full name and JobTitle
2. Write a LEFT JOIN between Production.Product and Production.ProductInventory — show all products and their Quantity on hand (NULL if no inventory record)
3. Find the total number of orders per customer in Sales.SalesOrderHeader using GROUP BY — show only customers with more than 5 orders (HAVING)
4. Create: CREATE TABLE dbo.TrainingLog (LogID INT IDENTITY PRIMARY KEY, InternName NVARCHAR(100), TaskCompleted NVARCHAR(200), CompletedDate DATE DEFAULT GETDATE()) — then INSERT your own row
5. UPDATE your row in TrainingLog to change the TaskCompleted value. Do it inside BEGIN TRAN and COMMIT it
6. Practice ROLLBACK: BEGIN TRAN → DELETE your row → SELECT to confirm it is gone → ROLLBACK → SELECT again to confirm it is back

DP-300 Alignment: Section 2.1 — Write queries using Transact-SQL

JOINs — Σύνδεση πινάκων
INNER JOIN → Επιστρέφει μόνο rows που υπάρχουν και στους δύο πίνακες.
LEFT JOIN → Επιστρέφει όλα τα rows του αριστερού πίνακα + NULL όπου δεν υπάρχει match στον δεξί.

DML Golden Rule
Πάντα βάζουμε WHERE σε UPDATE και DELETE.
Χωρίς WHERE → επηρεάζονται όλα τα rows.
Χρησιμοποιούμε Transaction για ασφάλεια:
BEGIN TRAN → ξεκινάει transaction.
COMMIT TRAN → αποθηκεύει αλλαγές.
ROLLBACK TRAN → αναιρεί αλλαγές.

T-SQL Commands

INNER JOIN tB ON tA.col = tB.col → μόνο matching rows.
LEFT JOIN → όλα τα αριστερά rows + NULLs.

Aggregations:
COUNT() → πλήθος
SUM() → άθροισμα
AVG() → μέσος όρος
MIN() / MAX() → ελάχιστο / μέγιστο
GROUP BY → ομαδοποίηση δεδομένων.
HAVING → φίλτρο σε groups.

WEEK 3

Building the Blueprint

Database Design, DDL, Indexes & Schemas

⏱ 5–10 hrs / week | 🛠 SSMS + Azure Portal

Learning Objectives

- Create, alter, and drop tables using CREATE TABLE, ALTER TABLE, DROP TABLE
- Implement constraints: PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, DEFAULT
- Understand clustered vs non-clustered indexes and when to use each
- Create schemas and understand their role in organising objects
- Create simple views and stored procedures

Key Concepts

Constraints keep data honest: PRIMARY KEY uniquely identifies each row. FOREIGN KEY enforces relationships. UNIQUE prevents duplicates. CHECK validates values before storage. DEFAULT provides a fallback when no value is supplied.

Indexes — how SQL Server finds data fast: Without an index, SQL Server reads every row (table scan). With an index it jumps directly to the data (index seek). The clustered index defines the physical row order — one per table. Non-clustered indexes are separate lookup structures — many per table.

Filtered unique index: A regular UNIQUE constraint applies to all rows. A filtered unique index (CREATE UNIQUE INDEX ... WHERE condition) applies the uniqueness rule only to rows matching the filter — e.g. only one approved application per pet.

Free Learning Resources

Resource	What You'll Learn
MS Learn: Create Tables with Constraints	~1 hour. PKs, FKs, UNIQUE, CHECK, DEFAULT with hands-on exercises.
WiseOwl: SQL Server Indexes Explained	20-min clear explanation of clustered vs non-clustered indexes. Watch before the lab. Search: 'WiseOwl SQL Server indexes'.
MS Learn: Design a Database Schema	Covers database design principles, normalisation, and schema creation.

🔬 Hands-On Lab — Build a Library Database from Scratch

Complete all tasks and send screenshots of your results to your mentor each week.

1. Create a new database: CREATE DATABASE LibraryDB; USE LibraryDB;
2. Create a schema: CREATE SCHEMA lib;
3. Create lib.Members: MemberID INT IDENTITY PK, FirstName NVARCHAR(100) NOT NULL, LastName NVARCHAR(100) NOT NULL, Email NVARCHAR(200) UNIQUE, JoinDate DATE DEFAULT GETDATE()
4. Create lib.Books: BookID INT IDENTITY PK, Title NVARCHAR(300) NOT NULL, Author NVARCHAR(200), ISBN NVARCHAR(20) UNIQUE, IsAvailable BIT DEFAULT 1
5. Create lib.Loans: LoanID INT IDENTITY PK, BookID FK → lib.Books, MemberID FK → lib.Members, LoanDate DATE NOT NULL, DueDate DATE NOT NULL, ReturnDate DATE NULL — plus CHECK constraint that DueDate > LoanDate
6. Add a non-clustered index on lib.Loans(MemberID) and another on lib.Books(Author). Then INSERT 3 books, 2 members, and 2 loans
7. Verify your schema using: SELECT * FROM sys.tables WHERE schema_id = SCHEMA_ID('lib') and SELECT * FROM sys.indexes WHERE object_id = OBJECT_ID('lib.Loans')

DP-300 Alignment: Section 1.2 — Implement databases and database objects

Constraints

* **PRIMARY KEY** → Μοναδικό ID για κάθε row (δεν επιτρέπει duplicates/NULL).
* **FOREIGN KEY** → Εξασφαλίζει σχέσεις μεταξύ πινάκων.
* **UNIQUE** → Αποτρέπει διπλές τιμές.
* **CHECK** → Ελέγχει αν οι τιμές είναι έγκυρες πριν αποθηκευτούν.
* **DEFAULT** → Βάζει προκαθορισμένη τιμή αν δεν δοθεί κάποια.

Indexes

* Χωρίς index → **Table Scan** (διαβάζει όλα τα rows).
* Με index → **Index Seek** (βρίσκει γρήγορα τα δεδομένα).
* **Clustered Index** → Καθορίζει τη φυσική σειρά των δεδομένων (1 ανά table).
* **Non-Clustered Index** → Ξεχωριστή δομή αναζήτησης (πολλά ανά table).

Filtered Unique Index

* UNIQUE constraint → ισχύει για όλα τα rows.
* Filtered Unique Index → ισχύει μόνο για rows που πληρούν μια συνθήκη.
* Παράδειγμα: μόνο **μία Approved αίτηση ανά Pet**.

WEEK 4

Lock It Down

Security — Logins, Users, Roles & Permissions

⏱ 5–10 hrs / week | 🛠 SSMS + Azure Portal

📌 Learning Objectives

- Understand SQL Server authentication modes (Windows Auth vs SQL Server Auth)
- Know the difference between a Login (server-level) and a User (database-level)
- Assign server roles and database roles appropriately
- Use GRANT, DENY, and REVOKE to control object-level permissions
- Apply the principle of least privilege to every user you create
- Create contained database users in Azure SQL (no login required)

📖 Key Concepts

Login vs User — a critical distinction: A Login lives at the SQL Server instance level and is how someone authenticates to the server. A User lives at the database level and controls what that person can do inside a specific database. One Login maps to one User per database.

Principle of Least Privilege: Every login/user should have only the minimum permissions needed for their job — nothing more. Never give everyone db_owner unless they genuinely need it.

DENY overrides GRANT: If a user is granted a permission through a role but explicitly denied the same permission directly, the DENY always wins — even if the grant came from a higher-level role.

Azure SQL difference: Azure SQL uses 'contained database users' — created directly in a database with a password, no server login required. This is the recommended approach for Azure SQL.

Key T-SQL:

- CREATE LOGIN lname WITH PASSWORD = 'Pwd!'
- CREATE USER uname FOR LOGIN lname (SQL Server)
- CREATE USER uname WITH PASSWORD = 'Pwd!' (Azure SQL — contained)
- ALTER ROLE db_datareader ADD MEMBER uname
- GRANT SELECT ON schema.Table TO uname
- DENY DELETE ON schema.Table TO uname

Το Login υπάρχει σε επίπεδο Server, ενώ ο User υπάρχει σε επίπεδο Database.
Το Login χρησιμοποιείται για την είσοδο στον SQL Server και ο User καθορίζει τα δικαιώματα μέσα σε συγκεκριμένη βάση δεδομένων.
Δεν δίνουμε ποτέ σε κάποιον db_owner permissions εκτός αν υπάρχει πραγματική ανάγκη ή περίπτωση έκτακτης ανάγκης, γιατί παρέχει πλήρη έλεγχο στη βάση.
Ακολουθούμε πάντα την αρχή των Minimum Required Permissions (Least Privilege Principle), δηλαδή κάθε χρήστης ή login πρέπει να έχει μόνο τα δικαιώματα που χρειάζεται για να εκτελεί τις εργασίες του και όχι περισσότερα.
Στον SQL Server, το DENY υπερισχύει του GRANT.
Αν σε έναν χρήστη δοθεί ταυτόχρονα ένα δικαίωμα μέσω GRANT και αφαιρεθεί μέσω DENY, το DENY θα έχει προτεραιότητα και η πρόσβαση θα απορριφθεί.

Free Learning Resources

Resource	What You'll Learn
MS Learn: Implement a Secure Environment	Core DP-300 security module. Logins, users, roles, permissions in SQL Server and Azure SQL.
TechBrothersIT: SQL Server Security Series	Comprehensive SQL Server security playlist. Watch the logins, users, and roles episodes. Search: 'TechBrothersIT SQL Server security'.
MS Learn: Configure Azure SQL Database Security	Contained users, Azure AD auth, firewall rules, and Transparent Data Encryption.

Hands-On Lab

Note: To change authentication mode in SSMS: right-click your server → Properties → Security tab → under Server authentication, select 'SQL Server and Windows Authentication mode' → OK → restart the SQL Server service.

Complete all tasks and send screenshots of your results to your mentor each week.

1. Ensure SQL Server is in Mixed authentication mode (see note above)
2. Create a SQL Server Login: CREATE LOGIN InternReadOnly WITH PASSWORD = 'Tr@ining2024!'
3. In AdventureWorks2019, create a database user for that login: CREATE USER InternReadOnly FOR LOGIN InternReadOnly
4. Add the user to the db_datareader role: ALTER ROLE db_datareader ADD MEMBER InternReadOnly
5. Test it: open a new SSMS connection as InternReadOnly (SQL auth). Try a SELECT (should work). Try an INSERT (should fail). Screenshot both results
6. Verify your security objects: SELECT name, type_desc FROM sys.server_principals WHERE name = 'InternReadOnly' and SELECT name, type_desc FROM sys.database_principals WHERE name = 'InternReadOnly'
7. In your Azure SQL sandbox: CREATE USER AzureIntern WITH PASSWORD = 'Az@Intern2024!' then GRANT SELECT ON SCHEMA::SalesLT TO AzureIntern. Test the connection.

DP-300 Alignment: Section 3 — Implement a secure environment (40% of the DP-300 exam)

ησιμοποιείται αυτόματα και δεν υποστηρίζονται log backups, το FULL, όπου όλες οι αλλαγές
backups, και το BULK_LOGGED, που μειώνει το log κατά τη διάρκεια bulk εργασιών. Για ένα σωστό restore χρειάζεται μια συνεχόμενη backup chain (Full → Differential → Log backups), γιατί αν χαθεί ένα
τό το Azure Portal. Το NORECOVERY κρατά τη βάση σε κατάσταση Restoring για να δεχτεί επιπλέον backups, ενώ το RECOVERY ολοκληρώνει τη διαδικασία και φέρνει τη βάση Online.

WEEK 5

Never Lose a Byte

Backup & Restore — Recovery Models & Disaster Recovery

⏱ 5–10 hrs / week | 🛠 SSMS + Azure Portal

📌 Learning Objectives

- Explain the three recovery models and when to use each (SIMPLE, FULL, BULK_LOGGED)
- Perform Full, Differential, and Transaction Log backups via T-SQL
- Restore a database using a correct recovery sequence
- Understand Point-in-Time Restore and the backup chain requirement
- Explain how Azure SQL Database handles backups automatically

📖 Key Concepts

The three recovery models:

Δεν υπαρχει ιστορικο ολων των συνναλαγων δεν υποστηριζει Transaction log backups
μπορει να κανει restore στο τελευταιο full ή diff backup δεν ειναι καταλληλο για production οπου χρειαζεται ελαχιστη απωλεια δεδομενων
ΧΡΗΣΗ test databases μικρες εφαρμογες

- SIMPLE — transaction log is automatically reused. Easy to manage, but you can only restore to the last backup. No log backups possible.
- FULL — every transaction is logged. You can restore to any point in time, provided you have an unbroken log chain. Use this for production databases.
- BULK_LOGGED — reduces log space during bulk operations. Rare in junior DBA work.

FULL ->Επιτρεπει point in time επαναφορα σε συγκεκριμενη στιγμη χρειαζεται ομως full->diff->και ολα τα logs μετα το full/diff απαιτειται unbroken chain log
BULK_LOGGED-> Ειναι συνδυασμος full and simple μειωνει το μεγεθος του transaction log σε μεγαλες bulk operations bulk insert index rebuilds ειναι για περιορισμενη χρηση!

Backup chain — never break it: A Point-in-Time Restore requires: Full backup + all Differential backups since + all Transaction Log backups since, with no gaps. A missing log file breaks the chain.

Azure SQL Database: Microsoft automatically takes Full backups weekly, Differential every 12–24 hours, and Transaction Log backups every 5–12 minutes. You cannot run BACKUP DATABASE in Azure SQL. Point-in-Time Restore is triggered from the Azure Portal.

Key T-SQL:

- ALTER DATABASE dbname SET RECOVERY FULL
- BACKUP DATABASE db TO DISK = 'path\full.bak' WITH INIT, COMPRESSION
- BACKUP DATABASE db TO DISK = 'path\diff.bak' WITH DIFFERENTIAL
- BACKUP LOG db TO DISK = 'path\log.bak' WITH INIT
- RESTORE DATABASE db FROM DISK = 'path\full.bak' WITH NORECOVERY, REPLACE
- RESTORE LOG db FROM DISK = 'path\log.bak' WITH RECOVERY

NORECOVERY vs RECOVERY: WITH NORECOVERY leaves the database in a 'Restoring' state — ready to receive more backups. WITH RECOVERY brings the database online but closes the restore sequence.

Free Learning Resources

Resource	What You'll Learn
MS Learn: Back Up and Restore SQL Server Databases	Core DP-300 module. All backup types, recovery models, and restore sequences.
TechBrothersIT: SQL Server Backup and Restore	Practical SSMS and T-SQL walkthrough. Watch before the lab. Search: 'TechBrothersIT SQL backup restore'.
MS Learn: Azure SQL PITR and Long-Term Retention	How Azure SQL automated backups work and how to trigger a Point-in-Time Restore.

Hands-On Lab — The Disaster Recovery Drill

Note: This is the most important lab in the Program. You will deliberately break a database and fix it. Take your time.

Complete all tasks and send screenshots of your results to your mentor each week.

1. Set AdventureWorks2019 to FULL recovery: ALTER DATABASE AdventureWorks2019 SET RECOVERY FULL
2. Take a Full backup: BACKUP DATABASE AdventureWorks2019 TO DISK = 'C:\Backups\AW_Full.bak' WITH INIT, COMPRESSION, STATS = 10 — create C:\Backups first if needed
3. Verify dbo.TrainingLog exists (IF OBJECT_ID('dbo.TrainingLog') IS NULL, recreate it from Week 2 Lab Step 4). Insert a new row — this is your 'important post-backup data'
4. Take a Transaction Log backup: BACKUP LOG AdventureWorks2019 TO DISK = 'C:\Backups\AW_Log1.bak' WITH INIT
5. Simulate disaster: DROP TABLE dbo.TrainingLog — note the exact time you do this
6. Restore: RESTORE DATABASE AdventureWorks2019 FROM DISK = 'C:\Backups\AW_Full.bak' WITH NORECOVERY, REPLACE; then RESTORE LOG AdventureWorks2019 FROM DISK = 'C:\Backups\AW_Log1.bak' WITH RECOVERY
7. Verify: SELECT * FROM dbo.TrainingLog — your row should be back. Screenshot the proof.

DP-300 Alignment: Section 4.2 — Recommend a HA/DR strategy for Azure SQL Database

WEEK 6

Data Wrangler

Importing CSV & Flat Files into SQL Server

⏱ 5–10 hrs / week | 🛠 SSMS + Azure Portal

Learning Objectives

- Use the SSMS Import Flat File Wizard for one-off manual imports
- Write BULK INSERT T-SQL to automate and script CSV imports
- Use the BCP command-line utility for exporting and importing large files
- Handle common import problems: line endings, NULL fields, encoding, duplicate rows
- Validate imported data with row counts and spot-check queries

Key Concepts

Three tools for flat-file imports:

- SSMS Import Flat File Wizard — click-through UI, good for one-off imports. Right-click database → Tasks → Import Flat File.
- BULK INSERT (T-SQL) — scriptable and repeatable. Best for automation. Requires the file to be accessible from the SQL Server machine.
- BCP utility — command-line, fastest for very large files, runs from PowerShell or Command Prompt.

Critical BULK INSERT gotchas:

- Windows CSV files use CRLF line endings: ROWTERMINATOR = '\r\n'
- Mac/Linux CSV files use LF only: ROWTERMINATOR = '\n'
- Check your file before importing — open in Notepad++ and look at the bottom-right status bar for 'Windows (CRLF)' or 'Unix (LF)'
- FIRSTROW = 2 skips the column header row — always include this
- Empty CSV fields do not automatically import as NULL — use KEEPNULLS or handle post-import with UPDATE

BULK INSERT template:

```
BULK INSERT dbo.Products FROM 'C:\data\products.csv' WITH (FIRSTROW=2,
FIELDTERMINATOR=',', ROWTERMINATOR='\r\n', TABLOCK,
ERRORFILE='C:\data\errors.txt')
```

Sample CSV — Save as C:\data\products.csv

```
ProductCode,ProductName,Category,UnitPrice,StockQty,DiscontinuedDate
P001,Wireless Ergonomic Mouse,Peripherals,34.99,250,
```

```
P002,Mechanical Keyboard RGB,Peripherals,109.99,80,
P003,27-inch 4K Monitor,Displays,399.00,45,
P004,USB-C Docking Station,Peripherals,149.99,120,
P005,Laptop Stand Aluminium,Accessories,49.99,300,
P006,Webcam HD 1080p,Peripherals,79.99,95,2024-06-30
P007,Cable Management Kit,Accessories,,500,
P008,Noise Cancelling Headset,Audio,199.99,60,
```

Free Learning Resources

Resource	What You'll Learn
MS Docs: BULK INSERT (T-SQL Reference)	Full syntax reference — bookmark this.
WiseOwl: Importing Data into SQL Server	Walkthrough of the SSMS Wizard and BULK INSERT. Search: 'WiseOwl import CSV SQL Server'.
MS Docs: BCP Utility	Full BCP reference for large-scale exports and imports from the command line.

Hands-On Lab

Complete all tasks and send screenshots of your results to your mentor each week.

1. Create: CREATE TABLE dbo.ImportedProducts (ProductCode NVARCHAR(10) PRIMARY KEY, ProductName NVARCHAR(200) NOT NULL, Category NVARCHAR(100), UnitPrice DECIMAL(10,2), StockQty INT, DiscontinuedDate DATE NULL)
2. Method 1 — SSMS Wizard: Right-click database → Tasks → Import Flat File → follow wizard for products.csv. SELECT * FROM dbo.ImportedProducts to verify.
3. TRUNCATE TABLE dbo.ImportedProducts. Method 2 — BULK INSERT: write and run the BULK INSERT T-SQL to reload the same CSV (use FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='\r\n' for Windows)
4. Fix the NULL: UPDATE dbo.ImportedProducts SET UnitPrice = 19.99 WHERE ProductCode = 'P007'
5. BCP export: from Command Prompt run: bcp dbo.ImportedProducts out C:\data\exported.csv -c -t, -T -S YourServerName
6. Verify: SELECT COUNT(*) — should be 8. Check DiscontinuedDate for P006 imported correctly as a DATE.

DP-300 Alignment: Section 5 — Migrate data and implement data movement

WEEK 7

Launch to the Cloud

Migrating On-Prem SQL Server to Azure SQL Database

⏱ 5–10 hrs / week | 🛠 SSMS + Azure Portal

📌 Learning Objectives

- Understand Azure SQL Database service tiers and when to use each
- Assess a database for Azure compatibility using the Data Migration Assistant (DMA)
- Migrate using BACPAC export and import — the quickest method
- Understand Azure Database Migration Service (DMS) as the production-grade alternative
- Use Azure Data Studio Migration Extension as a laptop-friendly migration option
- Execute a post-migration validation checklist

📖 Key Concepts

Azure SQL service tiers:

- General Purpose — best starting point. Balanced price/performance for most workloads.
- Business Critical — lowest latency, in-memory OLTP, built-in read replicas. Higher cost.
- Free Tier — 32 GB, 100,000 vCore-seconds/month. Your sandbox tier.

Two migration methods:

- BACPAC — exports schema and data to a single .bacpac file. Import to Azure SQL via SSMS or Portal. Best for smaller databases and non-live migrations. Requires downtime.
- Azure Data Studio Migration Extension — the recommended laptop-friendly tool. Handles schema, data, and some online migration scenarios. Easier than DMS for a home network.

Azure DMS note: Azure Database Migration Service requires network connectivity from Azure to your SQL Server (port 1433 open). This requires port forwarding through your home router — complex for a sandbox. Use Azure Data Studio Migration Extension instead.

Connection string change — on-prem to Azure:

On-prem: Server=localhost; Database=myDB; Integrated Security=true

Azure SQL: Server=yourserver.database.windows.net,1433; Database=myDB; User ID=admin; Password=pwd; Encrypt=true; TrustServerCertificate=false

Features not available in Azure SQL (check with DMA):

- SQL Server Agent Jobs — use Elastic Jobs or Azure Automation
- Linked Servers — use External Data Source

Key differences:
Use FQDN (fully qualified domain name)
Add port 1433 explicitly
Switch from integrated Windows auth->SQL auth(user/pass)
Enable encryption

- Windows Authentication — use Azure AD or SQL auth
- BACKUP DATABASE command — Azure manages backups automatically

Free Learning Resources

Resource	What You'll Learn
MS Learn: Migrate SQL Workloads to Azure SQL	Core DP-300 migration module. DMS, BACPAC, and assessment tools end-to-end.
Download: Data Migration Assistant (DMA)	Assess your SQL Server DB for Azure SQL compatibility issues before migrating.
MS Docs: Export a Database to a BACPAC File	Step-by-step BACPAC export and import guide.
Azure Data Studio Migration Extension	Install this for the laptop-friendly migration path — no port forwarding needed.

Hands-On Lab — Migrate LibraryDB to Azure

Note: You will migrate LibraryDB (built in Week 3) to Azure SQL Database. If you restored your sandbox between weeks, recreate LibraryDB first using your Week 3 DDL scripts.

Complete all tasks and send screenshots of your results to your mentor each week.

retired

1. Install DMA and run an assessment against LibraryDB. Review the compatibility report even if it shows no issues — get familiar with the tool output.
2. BACPAC Method: in SSMS right-click LibraryDB → Tasks → Export Data-tier Application → save the .bacpac file to your desktop
3. Import to Azure: in SSMS right-click your Azure SQL server → Import Data-tier Application → select the .bacpac file. Monitor the progress.
4. Verify data integrity: connect to the migrated database and run SELECT COUNT(*) on each of your three tables. Compare the numbers to your local sandbox.
5. Recreate ContractorUser (from Week 4) as a contained database user in Azure SQL: CREATE USER ContractorUser WITH PASSWORD = 'Contr@ct2024!'; then GRANT SELECT ON SCHEMA::lib TO ContractorUser
6. Post-migration check using catalog views: SELECT * FROM sys.tables and SELECT * FROM sys.indexes to confirm all objects migrated. SELECT name, type_desc FROM sys.database_principals to confirm your users.

7. Update a sample connection string: take the SQL Server format and rewrite it in Azure SQL format (FQDN, port 1433, Encrypt=true, TrustServerCertificate=false)

DP-300 Alignment: Section 5 — Perform migration (core DP-300 migration domain)

Το General Purpose είναι το προτεινόμενο service tier για τις περισσότερες εφαρμογές, προσφέροντας ισορροπία μεταξύ κόστους και απόδοσης.
Το Business Critical προσφέρει τη χαμηλότερη καθυστέρηση (latency), υποστηρίζει In-Memory OLTP και διαθέτει ενσωματωμένα read replicas, αλλά έχει υψηλότερο κόστος.
Το Free Tier παρέχει έως 32 GB αποθήκευσης και 100.000 vCore-seconds/μήνα, κατάλληλο για δοκιμές και εργαστήρια.
Για μεταφορά βάσης δεδομένων στο Azure υπάρχουν δύο βασικές μέθοδοι: το BACPAC, το οποίο εξάγει schema και data σε ένα αρχείο .bacpac και είναι κατάλληλο για μικρές βάσεις και offline migrations (απαιτεί downtime), και το Azure Data Studio Migration Extension, που είναι η προτεινόμενη λύση για προσωπικό υπολογιστή, καθώς μεταφέρει schema και data και υποστηρίζει ορισμένα online migrations.
Το Azure Database Migration Service (DMS) απαιτεί δικτυακή σύνδεση από το Azure προς τον SQL Server (ανοιχτή θύρα 1433 και port forwarding), γι' αυτό συνήθως δεν προτείνεται για οικιακά εργαστήρια.
Κατά τη μετάβαση από on-premises σε Azure SQL αλλάζει και το connection string, καθώς αντί για τοπικό server χρησιμοποιείται ο Azure SQL server με όνομα χρήστη, κωδικό και κρυπτογράφηση (Encrypt=true).
Τέλος, ορισμένες δυνατότητες του SQL Server δεν υποστηρίζονται στο Azure SQL, όπως τα SQL Server Agent Jobs, τα οποία αντικαθίστανται από λύσεις όπως Elastic Jobs ή Azure Automation.

WEEK 8

Speed Matters

Performance Monitoring, Query Store & Index Tuning

⏱ 5–10 hrs / week | 🛠 SSMS + Azure Portal

Learning Objectives

- Enable and use Query Store to identify regressed and slow queries
- Read a basic execution plan and identify table scans vs index seeks
- Use sys.dm_db_missing_index_details to find missing index recommendations
- Identify blocking sessions using sys.dm_exec_requests
- Use SSMS Activity Monitor for real-time performance monitoring

Key Concepts

Query Store — your query history: Query Store captures query execution statistics over time. It stores query text, execution plans, and runtime statistics so you can spot when a query suddenly gets slower (a 'plan regression'). Enable with: ALTER DATABASE db SET QUERY_STORE = ON

Reading execution plans:

- Index Seek — SQL Server jumps directly to the right rows using an index. Fast. Good.
- Table Scan / Clustered Index Scan — SQL Server reads every single row. Slow for large tables. Means an index is missing or cannot be used.
- Key Lookup — SQL Server uses a non-clustered index to find the row, then fetches extra columns from the clustered index. An extra trip. Can be eliminated by including those columns in the index (INCLUDE clause).

Missing index DMVs:

SELECT * FROM sys.dm_db_missing_index_details WHERE database_id = DB_ID() — lists columns SQL Server thinks would help

SELECT * FROM sys.dm_db_missing_index_group_stats — shows the estimated benefit of each missing index

Blocking:

SELECT session_id, blocking_session_id, wait_type, status FROM sys.dm_exec_requests WHERE blocking_session_id > 0 — find blocked sessions

Free Learning Resources

Resource	What You'll Learn

MS Learn: Monitor and Optimize Operational Resources	Core DP-300 performance module. Query Store, wait stats, Activity Monitor.
Brent Ozar: How to Read Execution Plans	Best free intro to execution plans on YouTube. Search: 'Brent Ozar read execution plans'.
MS Docs: Query Store Best Practices	Official guide to enabling and using Query Store effectively.

Hands-On Lab

Complete all tasks and send screenshots of your results to your mentor each week.

1. Enable Query Store: ALTER DATABASE AdventureWorks2019 SET QUERY_STORE = ON (QUERY_CAPTURE_MODE = AUTO, MAX_STORAGE_SIZE_MB = 100)
2. Run a table scan: press Ctrl+M in SSMS to enable Actual Execution Plan, then run: SELECT * FROM Sales.SalesOrderHeader — observe the scan
3. In SSMS: right-click AdventureWorks2019 → Reports → Standard Reports → Top Resource Consuming Queries. Screenshot the top result.
4. Run: SELECT * FROM sys.dm_db_missing_index_details WHERE database_id = DB_ID() — is there a recommended index? Note the columns suggested.
5. Create the suggested index (or create: CREATE NONCLUSTERED INDEX ix_test ON Sales.SalesOrderHeader(OrderDate) INCLUDE (TotalDue))
6. Re-run the same query with Actual Execution Plan. Did it switch from scan to seek? Screenshot the before and after plans.
7. Check for blocking: SELECT session_id, blocking_session_id, wait_type FROM sys.dm_exec_requests WHERE blocking_session_id > 0 — note the result (likely empty on your sandbox, which is fine)

DP-300 Alignment: Section 6 — Monitor and optimize operational resources (DP-300)

Το Query Store αποθηκεύει το ιστορικό των queries, τα execution plans και τα στατιστικά απόδοσης ώστε να εντοπίζονται αργά queries και Plan Regression (όταν ένα νέο execution plan είναι πιο αργό από το προηγούμενο).

Το Execution Plan δείχνει τον τρόπο με τον οποίο ο SQL Server εκτελεί ένα query. Το Index Seek χρησιμοποιεί index για να εντοπίσει γρήγορα τις ζητούμενες εγγραφές και είναι η πιο αποδοτική μέθοδος αναζήτησης, ενώ το Table Scan και το Clustered Index Scan διαβάζουν όλες τις γραμμές ενός πίνακα ή clustered index και είναι πιο αργά σε μεγάλους πίνακες.
Το Key Lookup συμβαίνει όταν ο SQL Server χρειάζεται να ανακτήσει επιπλέον στήλες που δεν υπάρχουν στο index και μπορεί να μειώσει την απόδοση· αποφεύγεται με τη χρήση του INCLUDE.
Οι Missing Index DMVs προτείνουν indexes που μπορούν να βελτιώσουν την απόδοση, ενώ οι Dynamic Management Views (DMVs) παρέχουν πληροφορίες για την κατάσταση και την απόδοση του SQL Server.
Το Blocking συμβαίνει όταν μία συνεδρία (session) κρατάει lock σε δεδομένα και μια άλλη περιμένει μέχρι να ολοκληρωθεί η πρώτη συναλλαγή.
Τέλος, τα Indexes χρησιμοποιούνται για τη βελτίωση της ταχύτητας των αναζητήσεων και μπορούν να δημιουργηθούν (CREATE INDEX) ή να διαγραφούν (DROP INDEX) ανάλογα με τις ανάγκες της βάσης δεδομένων.

WEEK 9

The Robot DBA

Automation — SQL Server Agent & Azure Elastic Jobs

⏱ 5–10 hrs / week | 🛠 SSMS + Azure Portal

Learning Objectives

- Create and schedule SQL Server Agent jobs with multiple steps
- Configure job schedules, operators, and email notifications on failure
- Create a maintenance plan for index rebuilds and statistics updates
- Understand Azure Elastic Jobs as the Azure SQL equivalent of SQL Agent
- Enable and configure Database Mail for job failure notifications

Key Concepts

Sql server agent μας βοηθαει στην αυτοματοποιηση καποιον tasks οπως backups και το cloud αδερφακι του ειναιτ το azure elastic jobs που λεγεται elastic γιατι μπορει να διαχειρηστει βασεις που ειναι elastic και κανουν Up/down scalling

SQL Server Agent: A Windows service that runs scheduled jobs. Each job has Steps (T-SQL, PowerShell, SSIS packages), Schedules (when to run), and optional Notifications (email on success/failure). The Agent must be running — right-click it in SSMS and start it if needed.

Database Mail: Allows SQL Server to send emails for alerts and notifications. Requires an SMTP server. For lab purposes, you can use a Gmail account with App Passwords or your organisation's SMTP relay.

Common maintenance jobs:

- Rebuild indexes (weekly) — fixes fragmentation that builds up from INSERT/UPDATE/DELETE
- Update statistics (nightly) — keeps the query optimiser's information accurate
- Backup databases (nightly/weekly) — full and differential on schedule
- Clean up old backup files — use FORFILES in a PowerShell step

Azure Elastic Jobs: The Azure SQL equivalent of SQL Agent. A serverless service that runs T-SQL jobs against one or many Azure SQL databases on a schedule. No SQL Agent service needed in Azure SQL — it runs in the background as a managed service.

Free Learning Resources

Resource	What You'll Learn
MS Learn: Automate Management Tasks with SQL Agent	Covers job creation, schedules, operators, and alerts. Core DP-300 automation module.

TechBrothersIT: SQL Server Agent Jobs	Practical walkthrough of creating Agent jobs. Search: 'TechBrothersIT SQL Server Agent'.
MS Docs: Elastic Jobs in Azure SQL	Official Elastic Jobs documentation — architecture and getting started guide.

Hands-On Lab

Complete all tasks and send screenshots of your results to your mentor each week.

1. In SSMS: right-click SQL Server Agent → Start (if not already running). Check it also starts automatically via SQL Server Configuration Manager.
2. Create a new Agent job: right-click Jobs → New Job. Name: 'Weekly_AW_Backup'
3. Add a T-SQL step: BACKUP DATABASE AdventureWorks2019 TO DISK = 'C:\Backups\AW_Weekly.bak' WITH INIT, COMPRESSION — name the step 'Run Full Backup'
4. Add a schedule: weekly, every Sunday at 2:00 AM. Name it 'Sunday Early Morning'
5. Right-click the job → Start Job at Step → run it now. Check Job History — screenshot the green success tick.
6. (Optional — requires SMTP relay) Enable Database Mail: EXEC sp_configure 'Database Mail XPs', 1; RECONFIGURE; then use the Database Mail Wizard in SSMS (Management → Database Mail) to create a Mail Profile. If you do not have an SMTP relay available, document the steps you would take and move to Step 7.
7. Review Azure Elastic Jobs: in the Azure Portal search for 'Elastic Job agent' — read through the overview page. Note: creation is not required in the free tier sandbox.

DP-300 Alignment: Section 7 — Automate database tasks using Azure resources

WEEK 10

Bulletproof Database

High Availability & Disaster Recovery

⏱ 5–10 hrs / week | 🛠 SSMS + Azure Portal

📌 Learning Objectives

- Define RTO (Recovery Time Objective) and RPO (Recovery Point Objective) and explain their trade-offs
- Describe SQL Server 2019 HA options: Always On AG, FCI, and log shipping
- Describe Azure SQL HA options: built-in HA, geo-replication, and failover groups
- Review Azure SQL backup retention settings and configure Long-Term Retention
- Design a basic HA/DR strategy for a given production scenario

📖 Key Concepts

RTO vs RPO — the fundamental HA trade-off:

- RTO (Recovery Time Objective) — how long can the service be down? e.g. 'We must be back online within 5 minutes'
- RPO (Recovery Point Objective) — how much data can we afford to lose? e.g. 'We can lose at most 1 minute of transactions'
- Shorter RTO and RPO = higher cost. There is always a trade-off between protection level and price.

SQL Server 2019 HA options (concepts):

- Always On Availability Groups (AG) — synchronise data between 2+ SQL Server instances. Synchronous mode = zero data loss but higher latency. Asynchronous mode = lower latency but some data loss on failover. Requires Windows Server Failover Cluster.
- Failover Cluster Instance (FCI) — shared storage cluster. The instance moves to another node on failure. Protects the instance, not individual databases.
- Log Shipping — automated backup/restore chain to a secondary server. Simple but manual failover.

Azure SQL HA options:

- Built-in HA — every Azure SQL database has 3 copies of data stored automatically. Zone redundancy adds copies across availability zones.
- Active Geo-Replication — async replication to a secondary database in another region. RPO ~5 seconds, RTO ~30 seconds. Manual failover only.
- Failover Groups — a group of databases that fail over together. Provides a listener endpoint (e.g. mygroup.database.windows.net) that auto-redirects after failover — applications do not need to change connection strings.

Long-Term Retention (LTR): Azure SQL can store backup copies for up to 10 years. Configure in the Azure Portal under your database → Backups → Configure policies.

Free Learning Resources

Resource	What You'll Learn
MS Learn: Plan and Implement HA/DR for Azure SQL	Core DP-300 HA/DR module. Geo-replication, failover groups, backup retention.
John Savill: Azure SQL HA Options Deep Dive	Excellent 30-min walkthrough. Search: 'John Savill Azure SQL High Availability'.
MS Docs: Always On Availability Groups Overview	Conceptual overview — you will not set this up on a single VM but you need to know it for DP-300.

Hands-On Lab

Complete all tasks and send screenshots of your results to your mentor each week.

1. In the Azure Portal: navigate to your Azure SQL Database → Backups. Review the backup retention settings and note the default PITR retention period.
2. Configure Long-Term Retention: Settings → Backups → Configure policies → set Weekly backups to be retained for 4 weeks. Save the policy.
3. Review geo-replication: navigate to your database → Data management → Replicas → review the options available (do not create a replica on the free tier — just review the UI and understand the configuration options).
4. In SSMS on your local sandbox: navigate to 'Always On High Availability' in the Object Explorer. This folder exists on all SQL Server instances but will be empty on a standalone server — this is expected and correct. Explore the sub-folders (Availability Groups, Availability Group Listeners) and read the right-click menu options to understand what options exist.
5. Research and write answers to these questions in a text file: (a) What is the RPO and RTO of BACPAC restore? (b) What is the RPO and RTO of Azure SQL PITR? (c) What is the RPO and RTO of a failover group with auto-failover?
6. Sketch a simple HA/DR architecture for a production Azure SQL database: show primary region, secondary region, failover group, and where the application connects.

DP-300 Alignment: Section 4 — Plan and implement a high availability and DR strategy

WEEK 11

Eyes on the Cloud

Azure Monitoring, Alerts & Operations

⏱ 5–10 hrs / week | 🛠 SSMS + Azure Portal

📌 Learning Objectives

- Use Azure Monitor Metrics to view database performance in the Azure Portal
- Create an alert rule that fires when a metric threshold is breached
- Use Intelligent Performance recommendations in Azure SQL
- Explore Query Performance Insight for Azure SQL
- Enable and understand Azure SQL Auditing

📖 Key Concepts

Azure Monitor: Azure's built-in monitoring platform. Collects metrics (DTU %, CPU %, storage, connections) from Azure SQL and can trigger alerts, send emails, or trigger Azure Functions when thresholds are breached.

Key Azure SQL metrics to watch:

- DTU percentage (DTU-based tiers) / CPU percentage (vCore/free tier) — how much compute capacity is being used. Sustained spikes above 80% mean you need to scale up or optimise queries.
- CPU percentage — compute load on the database server
- Data IO percentage — disk read/write load
- Connections count — number of active sessions
- Deadlocks — transactions that blocked each other and were killed by SQL Server

Intelligent Performance: Azure SQL automatically analyses query plans and recommends (and can even auto-apply) index additions and removals, query plan fixes, and schema recommendations. Found under your database → Intelligent Performance in the Portal.

Query Performance Insight: A Portal-based view of the top resource-consuming queries on your Azure SQL database, powered by Query Store data. No need to write DMV queries — it visualises the data for you.

Azure SQL Auditing: Records database access and modification events to a storage account, Log Analytics workspace, or Event Hub. Important for security compliance. Enable under Settings → Auditing.

Free Learning Resources

Resource	What You'll Learn
MS Learn: Monitor Azure SQL Database	Covers Azure Monitor, metrics, alerts, and Intelligent Performance.
MS Docs: Query Performance Insight	Official guide to using Query Performance Insight in the Azure Portal.
MS Docs: Azure SQL Auditing	How to enable, configure, and analyse Azure SQL audit logs.

Hands-On Lab

Complete all tasks and send screenshots of your results to your mentor each week.

1. Azure Portal: navigate to your Azure SQL Database → Monitoring → Metrics. Add a chart showing CPU percentage (vCore/free tier) or DTU percentage (DTU-based tiers) for the last hour. Screenshot it. Note: the free-tier sandbox uses serverless vCore pricing, so look for CPU percentage.
2. Create an alert: Monitoring → Alerts → New alert rule → condition = CPU percentage (or DTU percentage for DTU-based tiers) > 80% for 5 minutes → action = send email to yourself. Save the alert rule.
3. Explore Intelligent Performance: navigate to your database → Intelligent Performance → Performance recommendations. Screenshot the page whether or not there are recommendations.
4. Run a few queries against your Azure SQL database from SSMS (SELECT statements on larger tables). Wait 3–5 minutes, then in the Portal navigate to Intelligent Performance → Query Performance Insight. Screenshot the top queries chart. If it shows no data yet, wait a few more minutes and refresh — there is a processing delay before queries appear.
5. Enable Auditing: Settings → Auditing → toggle ON → select Log Analytics or Storage Account as the destination. Save the configuration.
6. Navigate to your Azure SQL server (not the database) → Security → Microsoft Defender for Cloud. Review what Defender for Cloud checks for and note the key recommendations shown.

DP-300 Alignment: Section 2.2 — Monitor activity and performance (DP-300)

WEEK 12

The Grand Finale

Full Capstone Project & DP-300 Countdown

⏱ 5–10 hrs / week | 🛠 SSMS + Azure Portal

🏆 Capstone Project — The Complete DBA Task

Scenario: You have been onboarded as the junior DBA for a small e-commerce startup. They have a SQL Server 2019 database on-premises and management wants it migrated to Azure SQL before go-live. Proper security and a backup strategy must be in place first. This is your first week on the job.

Phase 1 — Design & Build

Complete all tasks and send screenshots of your results to your mentor each week.

1. Create: CREATE DATABASE EcommerceDB on your local SQL Server
2. Create dbo.Customers: CustomerID INT IDENTITY PK, FullName NVARCHAR(200) NOT NULL, Email NVARCHAR(200) UNIQUE, City NVARCHAR(100), RegisteredAt DATETIME DEFAULT GETDATE()
3. Create dbo.Products: ProductID INT IDENTITY PK, Name NVARCHAR(300) NOT NULL, Category NVARCHAR(100), Price DECIMAL(10,2), Stock INT
4. Create dbo.Orders: OrderID INT IDENTITY PK, CustomerID FK → dbo.Customers, OrderDate DATE NOT NULL, TotalAmount DECIMAL(12,2), Status NVARCHAR(20) CHECK (Status IN ('Pending','Shipped','Delivered','Cancelled'))
5. Create a VIEW: CREATE VIEW dbo.vw_OrderSummary AS SELECT c.CustomerID, c.FullName, COUNT(o.OrderID) AS TotalOrders, SUM(o.TotalAmount) AS TotalSpent FROM dbo.Customers c LEFT JOIN dbo.Orders o ON c.CustomerID = o.CustomerID GROUP BY c.CustomerID, c.FullName
6. Create a stored procedure: CREATE PROCEDURE dbo.usp_GetOrdersByStatus @Status NVARCHAR(20) AS BEGIN SELECT * FROM dbo.Orders WHERE Status = @Status END
7. Insert at least 10 Customers, 8 Products, and 15 Orders with varied status values

Phase 2 — Security

Complete all tasks and send screenshots of your results to your mentor each week.

1. Create ReportingUser: a SQL login and DB user with SELECT on dbo.vw_OrderSummary only. Deny direct table access. Test it — screenshot a successful view query and a failed table query.
2. Create SalesAppUser: SELECT, INSERT, UPDATE on dbo.Orders and dbo.Customers. Deny DELETE on all tables. Test and screenshot.

3. Write a security design comment block: a SQL comment explaining who can do what and why — this is what you would present to your manager.

Phase 3 — Backup Strategy

Complete all tasks and send screenshots of your results to your mentor each week.

1. Set EcommerceDB to FULL recovery model. Run a Full backup to C:\Backups\Ecommerce_Full.bak
2. Write (as a comment in your script) a realistic backup schedule for a production e-commerce database: when would you run Full, Differential, and Log backups?
3. Take a Log backup. Delete 5 orders (BEGIN TRAN + COMMIT). Take another Log backup. Restore to the point before the deletions using your Full + first Log backup. Verify.

Phase 4 — Migration to Azure

Complete all tasks and send screenshots of your results to your mentor each week.

1. Run DMA against EcommerceDB. Document any compatibility findings.
2. Export EcommerceDB as a BACPAC. Import to your Azure SQL sandbox.
3. Recreate ReportingUser and SalesAppUser as contained database users in Azure SQL with the same permissions.
4. Verify: matching row counts, all objects exist (sys.tables, sys.views, sys.procedures), permissions work.

Phase 5 — Performance Optimisation

Complete all tasks and send screenshots of your results to your mentor each week.

1. Enable Query Store on EcommerceDB (both local and Azure).
2. Run SELECT * FROM dbo.Orders with Actual Execution Plan. Is it scanning or seeking?
3. Query sys.dm_db_missing_index_details — add any recommended indexes.
4. Re-run the query. Did performance improve? Screenshot before and after execution plans.

Phase 6 — Automation

Complete all tasks and send screenshots of your results to your mentor each week.

1. Create a SQL Agent job 'EcommerceDB_Nightly_Backup' that backs up EcommerceDB nightly at midnight.

2. Schedule it and execute it manually. Screenshot the successful Job History entry.

DP-300 Exam Prep Guide

The DP-300 exam validates your ability to manage SQL Server and Azure SQL databases. Your 12-week training Program covers all major exam domains.

Key exam facts:

- Format: multiple-choice questions plus performance-based lab tasks
- Duration: 120 minutes
- Passing score: 700 out of 1000
- Cost: approximately $165 USD — check the Microsoft website for current pricing and student discounts
- Validity: certified for 1 year; annual renewal required via Microsoft Learn

DP-300 Exam Domain	Weight	Covered In
Plan & implement data platform resources	**20–25%**	Weeks 1, 7
Implement a secure environment	**15–20%**	Week 4
Monitor & optimize operational resources	**15–20%**	Weeks 8, 11
Optimize query performance	**5–10%**	Week 8
Perform automation of tasks	**10–15%**	Week 9
Plan HA/DR strategy	**15–20%**	Weeks 5, 10
Administer by using T-SQL	**10–15%**	All weeks

Resource	What You'll Learn
MS Learn: DP-300 Official Study Path	The complete free official study path. Work through this after completing the 12 weeks.
John Savill: DP-300 Study Cram (Free, 3 hrs)	Excellent free cram session. Watch this the week before your exam. Search: 'John Savill DP-300 cram'.
MS Learn Sandbox Labs	Free browser-based Azure labs — no subscription needed. Great for exam practice.

Your recommended study path after completing this Program:

- 1. Complete all 12 weeks of this Program (done! ☑)
- 2. Work through the official MS Learn DP-300 path — focus on topics not covered in depth here (Elastic Jobs, advanced Query Store, DMV queries, geo-replication labs)
- 3. Watch John Savill's DP-300 cram video
- 4. Take at least 2 full practice exams before booking
- 5. Book the exam when you are consistently scoring 75%+ on practice tests
- 6. Pass the exam. Add 'Microsoft Certified: Azure Database Administrator Associate' to your LinkedIn. 🎓

T-SQL Quick Reference — Cheat Sheet

Querying

```
SELECT TOP 10 FirstName, LastName FROM Person.Person WHERE LastName LIKE 'S%' ORDER BY LastName ASC;
SELECT c.CustomerID, COUNT(o.OrderID) AS Orders FROM dbo.Customers c LEFT JOIN dbo.Orders o ON c.CustomerID = o.CustomerID GROUP BY c.CustomerID HAVING COUNT(o.OrderID) > 5;
```

DML — Modify Data

```
INSERT INTO dbo.Logs (Message, LogDate) VALUES ('Import complete', GETDATE());
UPDATE dbo.Products SET Price = Price * 1.05 WHERE Category = 'Electronics';
DELETE FROM dbo.Logs WHERE LogDate < DATEADD(DAY, -30, GETDATE());
BEGIN TRAN; DELETE FROM dbo.Products WHERE Stock = 0; SELECT COUNT(*) FROM dbo.Products; ROLLBACK;  -- or COMMIT
```

DDL — Build Objects

```
CREATE TABLE dbo.MyTable (ID INT IDENTITY(1,1) PRIMARY KEY, Name NVARCHAR(200) NOT NULL, Status NVARCHAR(20) CHECK (Status IN ('Active','Inactive')), CreatedAt DATETIME DEFAULT GETDATE());
ALTER TABLE dbo.MyTable ADD Email NVARCHAR(200) UNIQUE;
CREATE NONCLUSTERED INDEX ix_Name ON dbo.MyTable(Name) INCLUDE (Status);
CREATE UNIQUE INDEX ix_filtered ON dbo.Loans(PetID) WHERE Status = 'Approved';  -- filtered unique index
CREATE VIEW dbo.vw_Active AS SELECT * FROM dbo.Users WHERE IsActive = 1;
CREATE PROCEDURE dbo.usp_Get @ID INT AS BEGIN SELECT * FROM dbo.T WHERE ID = @ID; END;
```

Security

```
CREATE LOGIN AppUser WITH PASSWORD = 'Str0ng@Pass!';
CREATE USER AppUser FOR LOGIN AppUser;
CREATE USER AzureUser WITH PASSWORD = 'Az@Pass2024!';  -- Azure SQL contained user
ALTER ROLE db_datareader ADD MEMBER AppUser;
```

```
GRANT SELECT ON dbo.Products TO AppUser;  DENY DELETE ON dbo.Products TO AppUser;
SELECT name, type_desc FROM sys.server_principals WHERE name = 'AppUser';
SELECT name, type_desc FROM sys.database_principals WHERE name = 'AppUser';
```

Backup & Restore

```
ALTER DATABASE MyDB SET RECOVERY FULL;
BACKUP DATABASE MyDB TO DISK = 'C:\Backups\MyDB_Full.bak' WITH INIT, COMPRESSION, STATS = 10;
BACKUP DATABASE MyDB TO DISK = 'C:\Backups\MyDB_Diff.bak' WITH DIFFERENTIAL;
BACKUP LOG MyDB TO DISK = 'C:\Backups\MyDB_Log.bak' WITH INIT;
RESTORE DATABASE MyDB FROM DISK = 'C:\Backups\MyDB_Full.bak' WITH NORECOVERY, REPLACE;
RESTORE LOG MyDB FROM DISK = 'C:\Backups\MyDB_Log.bak' WITH RECOVERY;
```

BULK INSERT

```
BULK INSERT dbo.MyTable FROM 'C:\data\file.csv' WITH (FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='\r\n', TABLOCK, ERRORFILE='C:\data\errors.txt');
```

Note: Windows CSV files use ROWTERMINATOR='\r\n'. Mac/Linux files use '\n'. Check in Notepad++ before running.

System Views — Verify Your Work

```
SELECT * FROM sys.tables;                    -- all tables in current DB
SELECT * FROM sys.objects WHERE type = 'P';    -- all stored procedures
SELECT * FROM sys.indexes WHERE object_id = OBJECT_ID('dbo.MyTable');
SELECT * FROM sys.dm_db_missing_index_details WHERE database_id = DB_ID();
SELECT session_id, blocking_session_id, wait_type FROM sys.dm_exec_requests WHERE blocking_session_id > 0;
SELECT DB_NAME(); SELECT @@VERSION; SELECT GETDATE();
```